Práctica 3. Árboles AVL

Sesiones de prácticas: 2

Objetivos

Implementar la clase `AVL<T>` utilizando **patrones de clase y excepciones**. Programa de prueba para comprobar su correcto funcionamiento.

Descripción de la EEDD

Implementar la clase `AVL<T>` para que tenga toda la funcionalidad de un árbol equilibrado AVL en memoria dinámica, tal y como se describe en la Lección 11, utilizando patrones de clase y excepciones. Los métodos a implementar serán los siguientes:

En concreto, se usará:

- Constructor por defecto `AVL<T>()`
- Constructor copia (proceso en preorden) `AVL<T>(const AVL<T> &origen)`.
- Operador de asignación (=)
- Rotación a derechas dado un nodo `rotDer(Nodo<T>* &nodo)`
- Rotación a izquierdas dado un nodo `rotIzq(Nodo<T>* &nodo)`
- Operación de inserción `bool inserta(T& dato)`
- Operación de búsqueda recursiva `T* buscaRec(T &dato)`
- Operación de búsqueda iterativa `T* buscaIt(T &dato)`
- Recorrido en inorden[1] `VDinamico<T*> recorreInorden()`
- Número de elementos del AVL `unsigned int numElementos()`
- Altura del AVL, `unsigned int altura()`
- Destructor correspondiente (proceso en postorden)

Tanto el **constructor de copia** como el **operador de asignación** deben crear copias idénticas. El constructor se crea mediante un recorrido en **preorden** y el destructor se hace en **postorden**. El contador del **número de elementos** puede realizarse mediante un atributo contador o mediante un recorrido en O(n).

[1] Devuelve un vector dinámico de punteros a los elementos del AVL haciendo un recorrido en Inorden

Proyecto de prueba: Gestión de control de parques

En esta práctica se amplía el sistema *ControlParques*, desarrollado en las prácticas anteriores, para gestionar los ejemplares de árboles que contiene cada parque. En el siguiente UML se mantienen las clases *Especie* y *Parque* y se incorpora la nueva clase *Arbol*, que representa cada ejemplar mediante su identificador (*assetnum*), los datos del parque, distrito y barrio al que pertenece, sus coordenadas, perímetro y altura.

La clase principal *ControlParques* sigue almacenando las especies en un vector dinámico y los parques en una lista doblemente enlazada. Además, almacena los árboles en un árbol AVL, ordenados por su identificador *assetnum*, que permite realizar búsquedas eficientes de los ejemplares por esta clave.

Cada árbol está asociado a una *Especie* mediante una asociación. El fichero de árboles incluye el código de la especie de cada ejemplar, lo que permite establecer esta asociación automáticamente durante la carga de datos en el constructor de *ControlParques*. La clase *Arbol* incorpora los siguientes métodos:

- *Arbol::setEspecie():* establece la asociación del árbol con su especie.
- *Arbol::getCodEspecie():* devuelve el código de la especie asociada.
- *Arbol::getTipoEspecie():* devuelve el tipo de planta de la especie asociada.

La relación *Parque::contains* pasa a representar los árboles que contiene cada parque y se implementará mediante un vector dinámico de punteros a los objetos *Arbol* almacenados en el AVL. Además, se añaden los siguientes métodos:

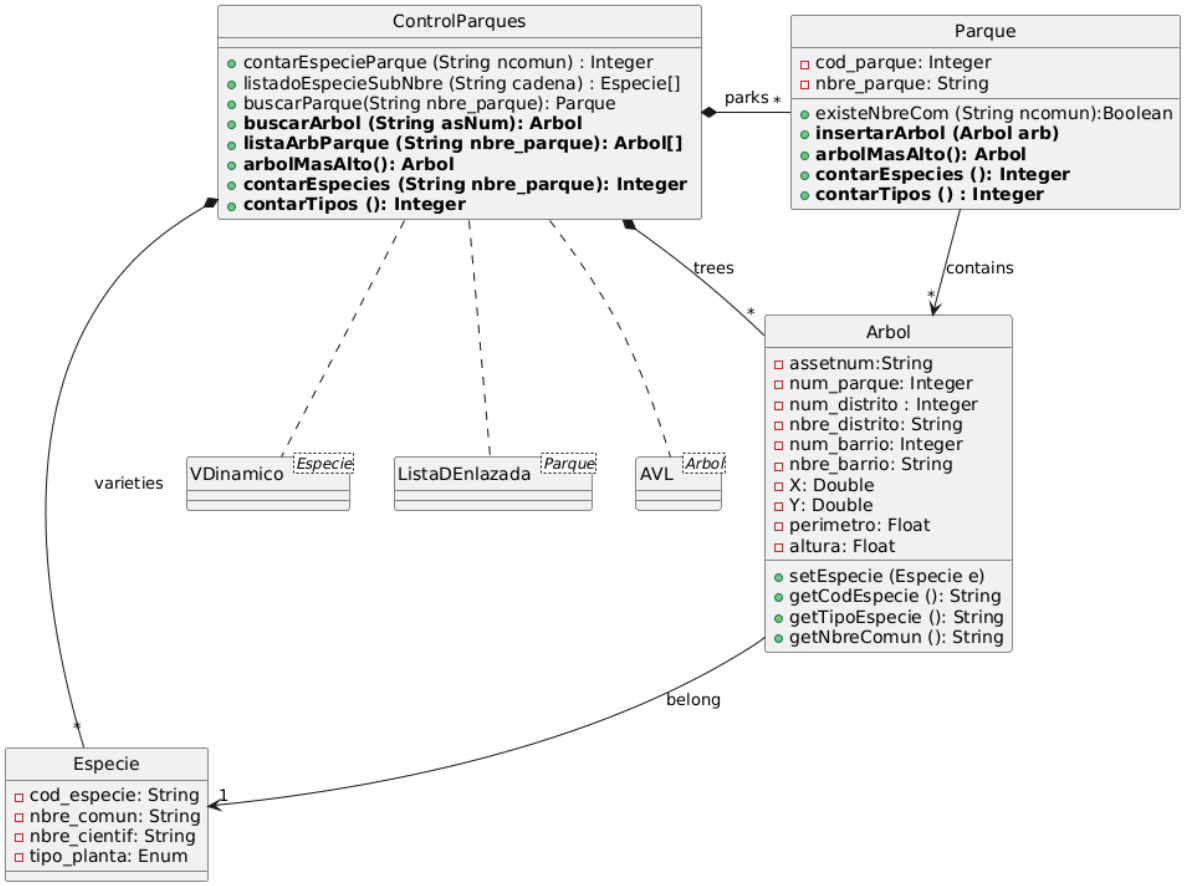
- *Parque::existeNbreCom():* aunque este método ya existía en la práctica anterior, hay que tener en cuenta que ahora la asociación de la clase *Parque* es con la clase *Arbol*, no con la clase *Especie*.
- *Parque:insertarArbol():* inserta un árbol en un parque.
- *Parque::arbolMasAlto():* devuelve el árbol de mayor altura del parque. Si hay empate, devolver el primero encontrado.
- *Parque::contarEspecies():* devuelve el número de especies distintas presentes en el parque.
- *Parque::contarTipos():* devuelve el número de tipos de planta distintos presentes en el parque.

Por último, la clase *ControlParques* incorpora nuevas operaciones de búsqueda, listado y recuento:

- *ControlParques::buscarArbol():* localiza un árbol en el AVL a partir de su identificador *assetnum*.
- *ControlParques::listaArbParque():* devuelve el listado completo de árboles de un parque dado su nombre.
- *ControlParques::arbolMasAlto():* devuelve el árbol de mayor altura de entre todos los parques. Si hay empate, devolver el primero encontrado.

- *ControlParques::contarEspecies():* devuelve el número de especies distintas de un parque dado su nombre.
- *ControlParques::contarTipos():* devuelve el número de tipos de planta distintos presentes en el conjunto de todos los parques.

El UML de la nueva funcionalidad se refleja a continuación:



Programa de Prueba I: probar la búsqueda de árboles de forma eficiente.

- Implementar el árbol *AVL<T>* según la especificación de la Lección 11.
- Adaptar el código de la lectura de ficheros anteriores para leer todos los árboles y añadirlos al árbol AVL del fichero *"arboladoMadrid.csv"*.
- Leer de nuevo todos los árboles y añadirlos también a un vector dinámico.
- Guardar en un vector auxiliar los identificadores de 500 árboles. Para seleccionar los 500 árboles, seleccionaremos uno de cada 250 elementos, a partir del primero, es decir, 0, 250, 500, 750…
- Buscar el *assetnum* de esos 500 árboles en el AVL y contabilizar el tiempo que se tarda en hacer esa búsqueda completa.
- Hacer la misma búsqueda lineal de los 500 datos en el vector dinámico y contabilizar el tiempo que se tarda en hacer esa búsqueda completa.

- Mostrar los tiempos en pantalla y determinar si se ha mejorado el tiempo de búsqueda con el uso del AVL.
- Mostrar la altura del árbol AVL.
- Recorrer el árbol en Inorden() y mostrar los 100 primeros árboles en pantalla.

Programa de prueba II: probar la nueva funcionalidad de ControlParques

A continuación se describen los pasos que debe seguir el **constructor de la clase ControlParques**, que tomará los nombres de los 3 ficheros como parámetros:

1. Leer el fichero de especies y almacenar los objetos *Especie* en el vector dinámico de *ControlParques*, como hasta ahora.
2. Leer el fichero de parques y almacenar los objetos *Parque* en la lista doblemente enlazada de *ControlParques*, como hasta ahora.
3. Leer el fichero «arboladoMadrid.csv» e insertar los árboles en el AVL, ordenados por *assetnum*. Asociar cada árbol con su especie mediante CODIGO_ESPECIE y establecer la asociación entre el parque correspondiente, identificado mediante NUM_PARQUE, y el árbol almacenado en el AVL.

Los ejercicios a resolver son los siguientes:

4. Mostrar el número de especies, parques y árboles cargados, así como la altura del AVL.
5. Buscar los árboles cuyos identificadores son 9850684, 5702331, 5697360 y 0000000. Mostrar su identificador, código de parque, nombre común, nombre científico y tipo de planta, si existe.
6. Obtener los árboles del parque «JARDINES DEL BUEN RETIRO». Mostrar el número de árboles y los datos de los diez primeros árboles del vector.
7. Buscar en el AVL los identificadores de los diez árboles anteriores y comprobar que los objetos devueltos son los mismos que están asociados al parque.
8. Contar las distintas especies presentes en los parques «JARDINES DEL BUEN RETIRO», «PARQUE DEL OESTE» y «QUINTA DE LOS MOLINOS». Mostrar el número de especies e indicar cuál contiene más especies distintas.
9. Mostrar el número de tipos de plantas distintas presentes en cada uno de los tres parques anteriores.
10. Buscar las especies “Pino piñonero” y “Olivilla” por su nombre común en el parque «JARDINES DEL BUEN RETIRO» y comprobar si existen.
11. Mostrar el número de tipos de plantas distintas presentes en el conjunto de todos los parques. Los valores vacíos no se contabilizarán como un tipo de planta.
12. Mostrar el identificador, la altura y la especie del árbol más alto que hay plantado en el conjunto de parques.

Para los que trabajan en parejas:

1. Implementar la función *ControlParques::buscarArbolesRango(asNumMin, asNumMax)*, que devuelve un *VDinamico<Arbol*>* con los árboles cuyo identificador *assetnum* está comprendido entre ambos límites, incluidos.
    - La búsqueda debe aprovechar la ordenación del AVL para evitar recorrer los subárboles que no puedan contener identificadores del intervalo. Añadir los métodos auxiliares necesarios a *AVL<T>*.
    - Si el intervalo no contiene árboles, se devolverá un vector vacío. Si el límite inferior es mayor que el superior, se lanzará una excepción.
    - Probar la función con el intervalo 5700000–5709999. Mostrar el número de árboles encontrados y los datos de los diez primeros resultados, o de todos si hay menos de diez.

IMPORTANTE:

- **Cuidado con realizar copias de objetos.**
- **Añadid *getters* y *setters* cuando se necesiten.**